

Seminar 5 exercises:

1. Create a procedure `display_employees` in which you declare a cursor to select the name, position and employment date of employees from the `Employees` table. Go through each row of the cursor and, if the employment date exceeds 01-AUG-2010, display the retrieved information. Call the procedure.
2. Create a function `seniority (p_employeeid employees.employee_id%TYPE)` that returns the employee's seniority (calculated as the difference between the current date and the employment date). Handle any exceptions that might occur. Call the function from a PL/SQL block and use a cursor to go through all employees.
3. Create a procedure `seniority2 (p_employeeid employees.employee_id%TYPE, seniority OUT number)` that calculates the seniority of the employee whose id is received as a parameter. Handle any exceptions that might occur. Call the procedure from a PL/SQL block and use a cursor to loop through all employees.
 1. Create a procedure `seniority3` that calculates the seniority of each employee (the records will be looped through a cursor). Handle the exceptions that might occur. Test the procedure.
 2. Create a procedure that returns the closing date and value of the most recent order: `infocomandarecenta (p_date OUT orders.orderdate%type, p_value OUT number)`.
 3. Create a trigger *positive quantities that does not allow negative values of the quantity that can be ordered in the orderitems table*. Test the trigger.
 4. Create a trigger `stop_increase` that prevents salary increases for employees with seniority greater than 10 years. Test the trigger.

Seminar 6 exercises

1. Create a package that includes:
a function that returns the number of orders placed by the customer whose ID is provided as a parameter. The function must handle the situation in which the specified customer does not exist by raising and managing an appropriate exception; a procedure that uses the previously defined function to retrieve the top 3 customers with the highest number of orders. The procedure must be invoked.

1. Create a package of subprograms that includes:
a procedure that inserts a new record into the `JOBS` table. The information to be inserted is provided as parameters to the procedure. The situation in which a job with the given code already exists must be handled as an exception; a procedure that updates the name of a job. The job code for which the update is performed and the new job title are provided as parameters to the procedure. The situation in which the update cannot be performed due to a non-existing job code must be handled as an exception; a procedure that deletes a job based on the code received as a parameter. The situation in which the provided code does not exist must be handled as an exception. The subprograms within the package must be invoked.

Seminar 7 exercises:

1. Create a PL/SQL procedure to increase by 20% the salary of commissioned employees who have handled at least 3 orders in a year, passed as a parameter. Return the number of employees for whom this update is being made, or handle an exception appropriately if there is no employee for whom the salary is being changed. Call the procedure and display the number of employees whose salary was changed.
2. Create a PL/SQL function to return the category in which an employee whose id is passed as a parameter falls. Employees with a salary of less than 3000 are junior, those with a salary between 3000 and 7000 are mid-level, and those with a salary over 7000 are senior. Handle the exception that occurs if the employee for whom the check is being made does not exist. (return an appropriate message).
3. Create a function that returns the full name of the employee given as a parameter. Handle the exception that occurs if the employee does not exist and call the function.

Seminar 8 exercises:

1. Display the name, position, and date of hire for those employees who joined the company in a given year, read from the keyboard.
 - If there is no such employee, handle the exception and display the message 'No new employees were hired in the year YYYY'
 - If the query returns multiple values, handle the exception and display the message 'Multiple employees were hired in the year YYYY', then display the list of those employees
 - Handle any other problem and display the message 'Another problem occurred'
1. Add a new department with ID 300 to the departments table, without specifying its name. In this case, an error with the code ORA-02290 will occur, warning us of an integrity constraint violation. Handle this situation by displaying an appropriate message, as well as the generated error message:
 2. Grant a 30 percent salary increase to an employee whose id you read from the keyboard and display a confirmation message. Handle the situation in which this employee does not exist with the help of an exception, displaying an appropriate message
 3. In a PL/SQL block, display the name of an employee whose id is read from the keyboard. Also, calculate the number of orders handled by the respective employee. Display the name of the employee, and if he exists and has handled orders, display their number. If the employee does not exist, handle the exception with an appropriate handling routine, and if the employee has not handled any orders, invoke an exception, which will be handled accordingly. Handle any other exception with an appropriate handling routine.

Seminar 10 exercises:

1. In a PL/SQL block, iterate over all employees with employee_id from 100 to 120, displaying their name, salary, and seniority.
- I2. In a PL/SQL block, iterate over all employees with an even employee_id, displaying their name, salary, and seniority
1. Write a PL/SQL block that reads a client_id from the keyboard and retrieves the client's information in a record-type variable, and then displays the client's data and the category in which they fall, knowing that:
 - credit_limit >= 10000 → "premium client";
 - credit_limit between 5000 and 9999 → "average client";
 - otherwise → "standard client";
 2. Using a PL/SQL block, assign a commission value to employees in the department whose id is read from the keyboard. Display the total number of changes made.
 3. Using a PL/SQL block, update the list price of products, based on their minimum price, as follows:
 - products with a minimum price between 1 and 100 receive a 25 percent increase in the list price
 - products with a minimum price between 101 and 1000 receive a 15 percent increase in the list price
 - other products do not have their prices increased.

Display the number of changes made.

1. Using a block, read a product category and a percentage increase from the keyboard, using substitution variables, and update the list price of products in that category. If no record has been modified, a corresponding message will be displayed, otherwise the number of updated products will be displayed.